Omni MCP — System Design (Go edition)

Oct 19, 2026 · @rohit · Implementation language: **Go**

This is the full v1.0 design, rewritten for Go. The first build is the much smaller MVP.

1. Overview

Omni MCP is a single MCP server that sits between an agent and many upstream MCP servers, exposing only four meta-tools so the agent can find and call any of thousands of tools without loading their definitions into context. It is modelled on the Omni MCP layer in Uber's MCP Gateway, built as a standalone Go service.

Problem. MCP has no cross-server search. An agent connected directly to 50 servers receives every tool schema up front. At a few hundred tokens per tool, 5,000 tools would cost over a million tokens, which no model can hold, and tool-selection accuracy drops well before that.

Goals

- One endpoint for the agent; any number of upstream MCP servers behind it.
- Constant agent-facing context: four meta-tools, regardless of how many tools exist.
- Accurate intent-based discovery across 10,000+ tools, proven by a public benchmark.
- Stability first: a failing upstream never breaks the agent or other upstreams.
- Transparent invocation: results look as if the agent called the upstream tool directly.

Non-goals for v1

- Converting gRPC/REST APIs into MCP tools (that is a gateway's job; plug Omni in front of one).
- Multi-tenant SaaS, billing, or a full admin UI.
- Hosting upstream servers; Omni connects to them, it does not run them (except spawning stdio processes).

Why Go. Goroutines and channels map directly onto "one supervised connection per upstream"; the standard library covers JSON, HTTP and process management; the result is a single static binary that is easy to ship and easy to read.

2. High-level architecture

Only the frontend is visible to the agent; every other component is internal, and each upstream server runs in its own isolated connection (its own goroutines).

Component	Responsibility	Section
MCP frontend	Exposes the four meta-tools, manages sessions	3
Upstream manager	Connects to and supervises every upstream server	4

Component	Responsibility	Section
Health checker	Probes servers, opens and closes circuits	5
Tool catalog	Stores servers, tools, schemas and usage	6
Search engine	Ranks servers and tools for a query	7
Invoke router	Validates, authorizes, forwards and trims calls	8
Security and policy	Access rules, approvals, secrets, redaction	10
Observability	Traces, metrics, audit log, benchmark	11

3. Agent-facing frontend and the four meta-tools

The frontend is an MCP server whose `tools/list` always returns exactly four tools, so the agent's context cost stays fixed at roughly 1–2k tokens. (MVP: hand-written JSON-RPC. Later: the official Go SDK, `github.com/modelcontextprotocol/go-sdk`.)

Transports. stdio for local agents (Claude Code, Cursor) and Streamable HTTP (`net/http`) for remote agents. Both share one core; the transport is just an adapter.

Sessions. Each connected agent gets a session holding its identity, its allowed servers (from policy), and a small cache of schemas it has already fetched.

Design rules for the meta-tools

- Descriptions are short and instruct the model on the order: discover, then schema, then invoke.
- `discover_tools` without `server` is a shortcut that lets strong models skip a step.
- Tool identity is always the pair `(server, tool)`, so name clashes between servers are impossible.
- Errors are returned as tool results with `isError: true` and a plain message plus a hint ("server payments is down; try discover_tools again"), never as protocol errors that would end the agent's turn.
- Optional later: a combined `search_and_schema` tool for clients that want fewer round trips.

Meta-tool	Input	Output	Purpose
`discover_server`	`query` (string), `limit` (int, default 5)	list of `{server, description, tool_count, health}`	Find servers matching an intent
`discover_tools`	`server` (optional), `query`, `limit` (default 10)	list of `{server, tool, one_line_description, score}`	Find tools inside one server, or across all if `server` is omitted
`get_tool_schema`	`server`, `tool`	full `inputSchema` + description + annotations	Load one schema only when needed
`invoke_tool`	`server`, `tool`, `arguments` (object), `fields` (optional list of paths)	the upstream `CallToolResult`, unchanged except projection	Execute the call

4. Upstream manager

The upstream manager owns one supervised connection per upstream server, so a slow or crashing server is isolated in its own goroutines and can never block the others.

Config (`omni.toml`; the MVP uses `omni.json`)

```
[[server]]
name        = "github"
transport   = "stdio"
command     = "npx"
args        = ["-y", "@modelcontextprotocol/server-github"]
env         = { GITHUB_TOKEN = "${GITHUB_TOKEN}" }
description = "Repos, issues, pull requests"
tags        = ["code", "git"]
timeout_ms  = 15000
max_concurrency = 8

[[server]]
name      = "logs"
transport = "http"
url       = "https://logs.internal/mcp"
auth      = { kind = "bearer", token = "${LOGS_TOKEN}" }
```

Connection (one supervisor goroutine per server)

1. Spawns the process (`os/exec`, stdio) or opens the HTTP session (Streamable HTTP).
2. Runs MCP `initialize`, records server info and capabilities.
3. Holds a bounded request queue; `max_concurrency` is a semaphore (buffered channel) so one agent cannot flood a server.
4. Restarts with exponential backoff (1s, 2s, 4s… capped at 60s) if the process exits or the session drops.
5. Publishes state changes (connected, reconnecting, failed) on an internal event channel.

Pools. stdio servers get one process by default (MCP stdio is a single stream); HTTP servers can use a pool of sessions. Pool size is per-server config.

Hot reload. Watching `omni.toml` (via `fsnotify`) lets servers be added or removed without restarting Omni. Removed servers are drained: in-flight calls finish, new calls are refused (`context` cancellation + `sync.WaitGroup`).

Auth to upstreams. Static tokens from env or a secrets file in v1. Later: per-user token passthrough, where the agent's identity token is exchanged for an upstream token, as Uber does for third-party servers.

5. Health checker and circuit breaker

Each server moves through five health states, and the state directly controls both search ranking and whether calls are allowed, so a broken server disappears from the agent's view within seconds.

Probes. Every 15–30 s (jittered so servers are not all pinged at once), Omni sends MCP `ping`. If a server does not support ping, a cheap `tools/list` is used instead. Probe timeout is short (e.g. 3 s, via `context.WithTimeout`).

Signals that feed the state

- Probe success or failure and its latency.
- Real call outcomes from the router: timeouts and transport errors count; a tool returning `isError: true` does not, because that is a normal tool answer.
- Process exits and dropped HTTP sessions from the connection supervisor.

Transitions

From	To	When
Healthy	Degraded	p95 latency above threshold, or error rate above 20% over the last minute
Degraded	Healthy	Metrics back under thresholds for 2 minutes
Degraded or Healthy	Down	3 consecutive failed probes or calls
Down	Half-open	Backoff expires (30 s, doubling to 5 min)
Half-open	Healthy	One probe succeeds
Half-open	Down	The probe fails; backoff doubles

Thresholds are per-server config. All values above are starting defaults to tune with real data.

Exposed state. `/health` returns Omni's own status plus each server's state; `discover_server` includes `health` in results so the agent can avoid a degraded server.

6. Tool catalog (registry)

The catalog is the single source of truth for which servers and tools exist, kept in SQLite on disk and mirrored in memory for reads that take microseconds. (MVP: memory only.)

Data model

Table	Key columns	Notes
`servers`	`name`, `description`, `tags`, `transport`, `health`, `last_seen`, `tool_count`	One row per upstream

Table	Key columns	Notes
`tools`	`server`, `name`, `title`, `description`, `input_schema` (JSON), `output_schema`, `annotations`, `schema_hash`, `enabled`, `updated_at`	Primary key `(server, name)`
`tool_aliases`	`server`, `tool`, `alias`	Extra search terms added by humans
`tool_versions`	`server`, `tool`, `schema_hash`, `seen_at`	History, for debugging schema drift
`usage_stats`	`server`, `tool`, `calls`, `errors`, `p50_ms`, `last_called`	Feeds ranking

Sync loop

1. On connect, call `tools/list` (paginating through `nextCursor`) and upsert every tool.
2. Compute `schema_hash` = SHA-256 of the canonical JSON schema; unchanged hash means no re-indexing.
3. Tools that disappeared are marked `enabled = false`, not deleted, so history and stats survive.
4. Refresh on the server's `notifications/tools/list_changed`, and also on a timer (default every 5 minutes) for servers that never send it.
5. Each change emits an event that the search engine consumes to update its indexes.

Enablement. New tools can be auto-enabled or start disabled until approved (`auto_enable = false` per server), following Uber's rule that discovery does not imply exposure.

Description enrichment (optional). If a tool's description is too short, an offline job asks an LLM to write a better one from the name and schema. The original is kept; the enriched text is used only for search.

7. Search engine

Search quality decides whether Omni works at all, so it uses hybrid retrieval: BM25 keyword search plus embedding similarity, merged with Reciprocal Rank Fusion and lightly re-weighted by health and usage. (MVP: a simple BM25-style ranker in `search.go`.)

What gets indexed per tool (one "search document")

- Server name, server description, tags
- Tool name split into words (`get_pull_request` → "get pull request")
- Title and description (original + enriched)
- Parameter names and their descriptions from the schema
- Human aliases

Keyword index. `bleve` (Go's full-text search library) with BM25 scoring and field boosts: tool name ×3, description ×2, params ×1. Catches exact terms such as "jira" or "invoice".

Semantic index. A small local embedding model (e.g. bge-small, 384 dimensions) run via ONNX Runtime (`onnxruntime_go`), or an optional local embedding endpoint such as Ollama. Vectors live in memory as

`[]float32`; for 10,000 tools that is about 15 MB, so brute-force cosine search takes around a millisecond. Swap in an HNSW index past roughly 100k tools.

Fusion and ranking

```
score(t) = Σ over r in {bm25, vec} of 1 / (k + rank_r(t))
           × h(t) × (1 + α · log(1 + calls(t)))
```

- `k` = 60 (standard RRF constant).
- `h(t)` = 1 for healthy, 0.5 for degraded, 0 for down (down servers drop out).
- `α` is small (around 0.05) so popularity breaks ties but never overrides relevance.

Server search vs tool search. `discover_server` ranks servers by the best and average scores of their tools plus their own description, so a server matches even when the query only fits one of its tools. `discover_tools` filters to one server, then ranks.

Optional re-ranker (v2). A small cross-encoder re-scores the top 20 results for higher precision, at a few extra milliseconds.

Index updates. Catalog events update both indexes incrementally; a full rebuild runs only at startup.

8. Invoke router

The router turns one `invoke_tool` call into one upstream `tools/call`, applying checks in a fixed order so every failure returns a clear, recoverable error to the agent.

Pipeline for each call

1. Resolve `(server, tool)` in the in-memory catalog; unknown tool → error with the closest matches from search.
2. Authorize against the session's policy (section 10).
3. Validate arguments against the cached JSON schema (`github.com/santhosh-tekuri/jsonschema`, compiled once per `schema_hash`). Invalid input → error listing the failing fields, so the model can fix and retry. (MVP: only `required` fields are checked.)
4. Health gate: if the circuit is open, fail fast with "server unavailable" instead of waiting.
5. Acquire a slot from the server's concurrency semaphore (queue wait capped, e.g. 2s).
6. Forward `tools/call` through the connection with a per-call deadline (`context.WithTimeout`; server `timeout_ms`, overridable per tool).
7. Project the response if `fields` was given (below).
8. Redact sensitive values if redaction rules match.
9. Record metrics and an audit log line, then return.

Retries. Only for transport failures before the request reached the server, and only for tools annotated `readOnlyHint` or `idempotentHint`. Never retry a tool that may have side effects; return the error and let the agent decide.

Cancellation. If the agent cancels, Omni cancels the call's `context`, forwards `notifications/cancelled` upstream and frees the slot.

Response projection. Borrowed from Uber: the agent passes `fields`, a list of paths such as `["items[].id", "items[].status"]`. Omni parses JSON in the result's text or `structuredContent` and keeps only those paths. Non-JSON results pass through untouched. Large results above a size limit (e.g. 50 KB) are truncated with a note telling the agent to use `fields`.

Large payloads. Results are kept as `json.RawMessage` and passed through without re-encoding where possible, keeping the proxy's own overhead under a millisecond.

9. Request lifecycle end to end

A typical task takes four agent calls, and only the last one reaches an upstream server; the first three are answered from Omni's own memory in milliseconds.

Example task: "find error logs for the payments service in the last hour."

1. `discover_server` — the search engine ranks servers; `logs` comes first with its health state.
2. `discover_tools` — ranks tools inside `logs`; the agent sees names and one-line descriptions only.
3. `get_tool_schema` — returns the full schema for `query_logs` from the catalog cache, no upstream call.
4. `invoke_tool` — the router validates arguments, checks policy and health, calls the upstream with a deadline, trims the result to the requested `fields`, redacts, and returns.

Strong models often skip step 1 by calling `discover_tools` without a server. Omni caches schemas per session, so repeat calls to the same tool go straight to step 4.

10. Security

Because every call goes through one generic `invoke_tool`, the client can no longer tell a harmless read from a destructive write, so Omni must enforce access and approval itself.

Identity. stdio sessions are trusted as the local user. HTTP sessions require a bearer token or OAuth 2.1 (as the MCP spec defines for remote servers); the token maps to a principal (user, service or agent).

Authorization policy (`policy.toml`)

```
[[rule]]
principal = "agent:support-bot"
allow     = ["jira/*", "logs/query_*"]
deny      = ["*/delete_*"]

[[rule]]
principal = "user:*"
allow     = ["*"]
require_approval = ["*/*"]   # only for tools without readOnlyHint
```

Rules are evaluated per `(server, tool)`, deny wins over allow. Discovery is filtered too: a principal never sees tools it cannot call, which also shrinks search results.

Approvals for risky tools. Tools with `destructiveHint` or no `readOnlyHint` can require confirmation. Omni uses MCP elicitation where the client supports it, otherwise returns an error asking the agent to confirm with the user and retry with `confirm: true`.

Secrets. Upstream tokens come from env vars or a secrets file, never from the agent. They are held in a small `Secret` type whose `String()`, `GoString()` and `MarshalJSON()` methods print `[REDACTED]`, so they never appear in logs or debug output.

Redaction. Regex and pattern rules (emails, card numbers, API keys) applied to results before they reach the model, configurable per server.

Prompt-injection defence. Tool descriptions and results come from upstreams and may contain instructions. Omni strips control characters, caps description length in search results, and can flag a server as untrusted so its text is wrapped and labelled as data.

Process isolation. stdio servers run as child processes with a clean environment (only their configured env vars) and, on Linux, optionally under a separate user or sandbox.

11. Observability and benchmark harness

Every meta-tool call is traced end to end, and a benchmark suite turns search quality from a claim into a number that is checked on every change.

Tracing. `log/slog` for structured logs + OpenTelemetry (`go.opentelemetry.io/otel`) export. One span per meta-tool call, child spans for search, validation, queue wait and upstream call. A `trace_id` is returned in result metadata for debugging.

Metrics (Prometheus `/metrics`, `client_golang`)

Metric	Type	Labels
`omni_meta_calls_total`	counter	`tool`, `outcome`
`omni_upstream_latency_seconds`	histogram	`server`, `tool`

Metric	Type	Labels
`omni_upstream_errors_total`	counter	`server`, `kind` (timeout, transport, tool_error)
`omni_server_health`	gauge	`server`, `state`
`omni_search_latency_seconds`	histogram	`index`
`omni_catalog_tools`	gauge	`server`
`omni_tokens_returned`	histogram	`tool`

Audit log. JSON lines: time, principal, server, tool, argument hash (not raw values), outcome, latency. Raw arguments only if explicitly enabled.

Discovery analytics. Log which tools were found, which were invoked after being found, and queries that found nothing. Queries with no follow-up invoke are the best signal of bad search and the input for improving descriptions.

Benchmark harness (`cmd/omni-bench`)

- Dataset: a set of real MCP servers (GitHub, filesystem, Slack, Postgres…) plus synthetic tools to scale the catalog to 1k, 5k and 10k tools.
- Queries: hundreds of natural-language tasks, each labelled with the correct `(server, tool)`, including hard cases with near-duplicate tools.
- Retrieval metrics: Recall@1, Recall@5, MRR, for BM25 only, embeddings only and hybrid.
- End-to-end: run a real agent on the tasks, measure success rate, round trips and total tokens, against the baseline of all tools loaded directly.
- Runs in CI (`go test` / GitHub Actions); a drop in Recall@5 fails the build.

12. Scaling, performance targets and failure modes

A single Omni process should comfortably handle 10,000 tools and hundreds of concurrent agents (goroutines are cheap); beyond that, Omni scales horizontally because its only shared state is the catalog.

Performance targets (design goals, to verify with the benchmark)

Measure	Target
Agent-facing context for tool definitions	under 2,000 tokens, constant
`discover_*` latency at 10k tools	p99 under 20 ms
Proxy overhead added to `invoke_tool`	p99 under 2 ms
Memory at 10k tools (catalog + indexes + model)	under 300 MB
Search Recall@5 on the benchmark	90% or higher

Horizontal scaling. Run several Omni instances behind a load balancer with sticky sessions (MCP sessions are stateful). Move the catalog from SQLite to Postgres + pgvector (`pgx`), with one instance elected as the sync

leader so upstreams are not polled N times.

Failure modes and responses

Failure	What Omni does
Upstream process crashes	Supervisor restarts it with backoff; calls fail fast while the circuit is open
Upstream hangs	Per-call deadline fires, slot released, call returns a timeout error
Upstream changes a schema mid-session	`list_changed` triggers re-sync; validation uses the new hash; agent gets a "schema changed, fetch again" error if it used the old one
Search returns nothing useful	Return the top server list and a hint to rephrase; log the query for analysis
Agent sends invalid arguments	Validation error naming the bad fields; agent retries
Omni itself restarts	Catalog reloads from disk in seconds; agents reconnect; no upstream state is lost
Embedding model fails to load	Fall back to BM25 only and report degraded search in `/health`
Huge tool result	Truncate with a note to use `fields` projection

13. Go package layout, tech stack and build roadmap

Omni is one Go module with small packages, so each layer can be tested alone and the search package can be reused as a library by other gateways.

```
omni-mcp/
├── cmd/
│   ├── omni/            # main binary
│   └── omni-bench/      # benchmark harness
├── internal/
│   ├── core/            # shared types: ServerID, ToolRef, errors, events
│   ├── upstream/        # connections, transports, health, circuit breaker
│   ├── catalog/         # SQLite store, sync loop, schema hashing
│   ├── router/          # validation, policy, projection, redaction, invoke
│   └── server/          # MCP frontend, the four meta-tools, sessions
├── pkg/
│   └── search/          # bleve + embeddings, fusion, ranking (importable by others)
├── omni.toml
└── policy.toml
```

(The MVP keeps everything in `internal/omni/`; see MVP_DESIGN.md.)

Tech stack

Need	Go choice
Concurrency	goroutines, channels, `context`, `sync`
MCP protocol	hand-written JSON-RPC (MVP) → `github.com/modelcontextprotocol/go-sdk`
HTTP	`net/http`
Keyword search	`bleve` (MVP: in-house ranker)
Embeddings	`onnxruntime_go` or a local Ollama endpoint
Storage	`database/sql` + `modernc.org/sqlite` (pure Go; later `pgx` for Postgres)
Schema validation	`santhosh-tekuri/jsonschema`
Config	`encoding/json` (MVP), `BurntSushi/toml`, `fsnotify`
Observability	`log/slog`, OpenTelemetry, `prometheus/client_golang`
Secrets	custom `Secret` type
Tests	`go test`, table-driven tests, `-race`

Roadmap

1. **v0.1 Pass-through (MVP).** Frontend + upstream + router with simple search. Proves the proxy works with real servers. *(Code written; needs real-server testing.)*
2. **v0.2 Catalog.** SQLite store, sync loop, `list_changed` handling, full schema validation.
3. **v0.3 Search.** bleve BM25 first, then embeddings and fusion; `discover_server` ranking.
4. **v0.4 Health.** Health checker, circuit breaker, auto-restart, hot reload.
5. **v0.5 Benchmark.** `omni-bench` with Recall@k and token savings; publish the first results.
6. **v0.6 Security.** Policy, approvals, redaction, HTTP auth.
7. **v1.0** Projection, metrics, docs, Docker image, stable config format.

Learning tip: v0.1 alone teaches Go concurrency, the MCP protocol and process management, which is most of the hard part.